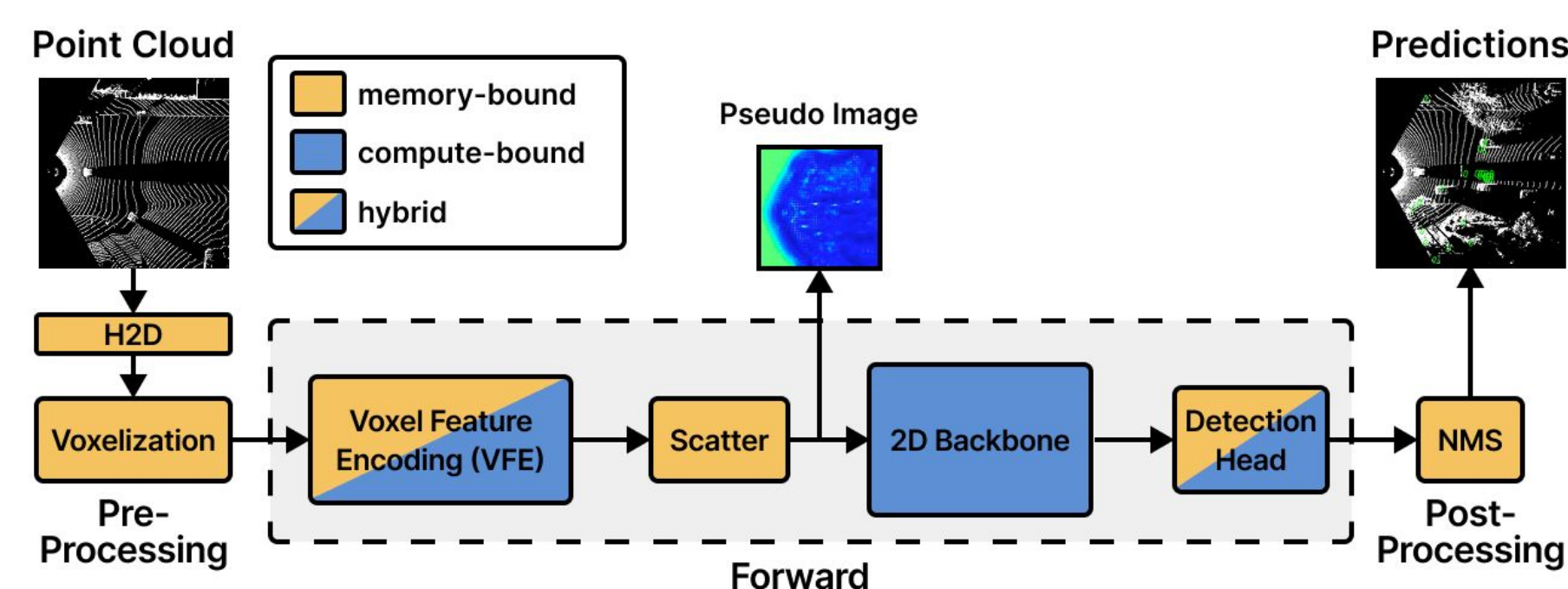


Motivation

Deploying 3D Perception at Edge Systems is Hard

- Autonomous driving requires real-time 3D object detection from LiDAR point clouds (**sparse, irregular memory patterns**)
- Edge GPUs have strict latency and energy budgets
- Native PyTorch (OpenPCDet) suffers from:
 - Heavy Python dispatch overhead
 - Frequent small kernel launches
 - Inefficient memory access patterns
 - CPU-bound voxelization bottleneck



Key Question: How software-level optimizations in 3D perception close the gap to hardware limits without sacrificing accuracy?

Related Work

- OpenPCDet (Shi et al., 2020)
- TorchSparse (Tang et al., 2022)
- CUDA-PointPillars (NVIDIA, 2022)
- Zeus (You et al., 2023)
- torch.compile (Ansel et al., 2024)
- SpConv (Yang et al., 2020)

Method

Our Progressive Optimization Stack for PointPillars

Method	Description
M0 Baseline	OpenPCDet implementation
M1 + Compiled	Graph-level fusion and overhead reduction with torch.compile
M2 + NHWC	Channel-last layout for Scatter and Conv2D
M3 + Voxel. Offload	Voxelization offloaded from host to GPU
M4 All Combined	Integration of M1–M3
M5 TensorRT	Industry-used CUDA-PointPillars

Operator Fusion & Overhead Reduction (M1)

- CUDA kernel launches: **−40%** (16,752 -> 10,014)
- Device-to-device memory traffic: **−85%** (4,472 MB -> 632 MB)

GPU Preprocessing (M3): Offloads CPU voxelization onto GPU

- Preprocessing latency: **3X faster**
- Eliminates ~20 MB of redundant host-to-device transfers

Mixed Precision (AMP): Enables FP16 Tensor Core execution across 2D backbone and anchor head

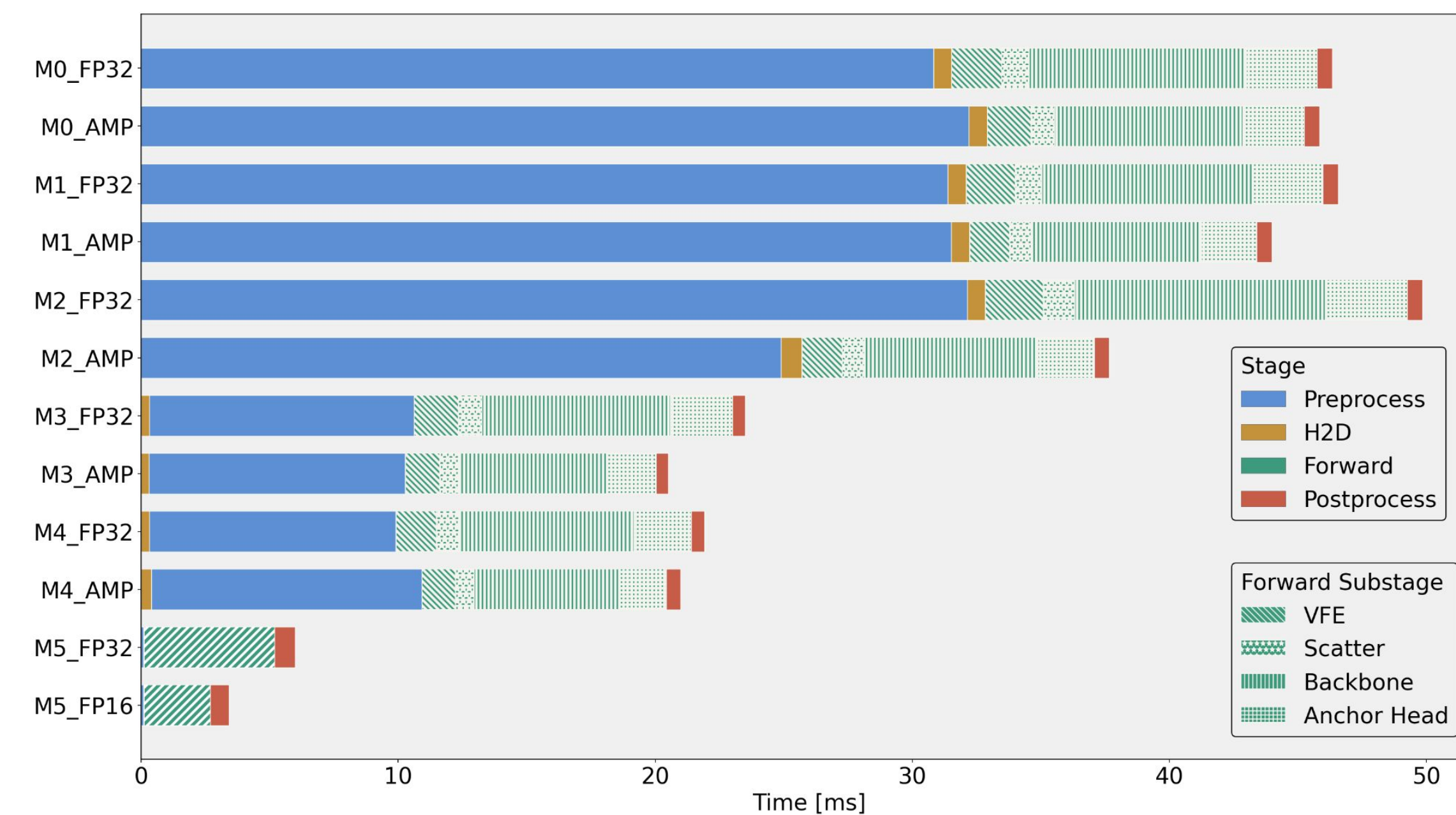
Evaluation

Accuracy (3D Detection, KITTI Dataset)

Precision	Car@R11 mAP	Car@R40 mAP
FP32	84.81	81.46
AMP	84.79	81.45

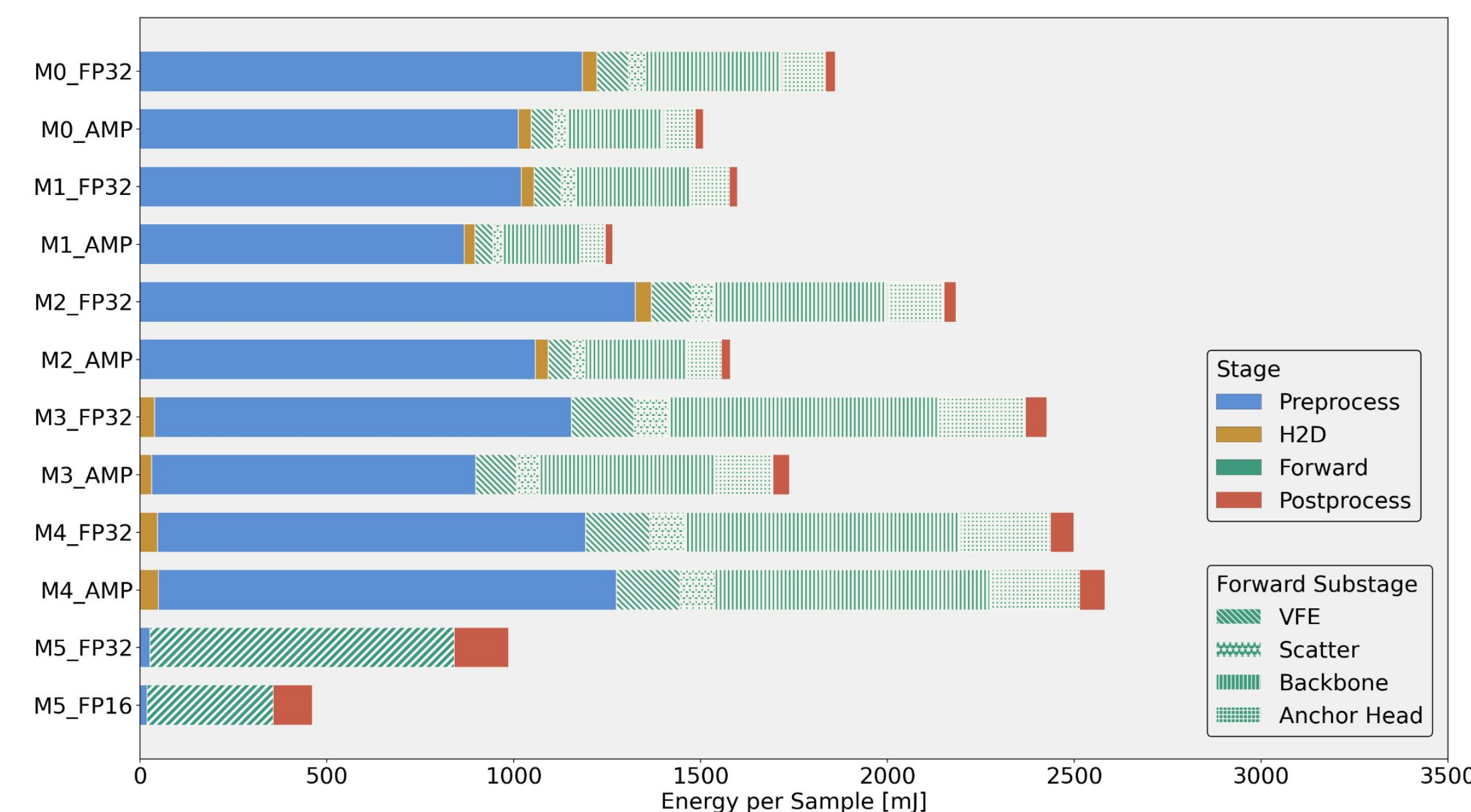
- No significant degradation

Latency (NVIDIA A10)



- Profiled with NVIDIA Nsight Systems
- M3 (Voxelization offloaded) makes largest gain (15.71 ms -> 10.51 ms)
- M2 + FP32 (17.88 ms) regresses for layout transform cost
- M2 + AMP (12.95 ms) enables FP16 Tensor Core computation
- M4 (Combined): 10.37 ms AMP, closing to TensorRT (3.34 ms)

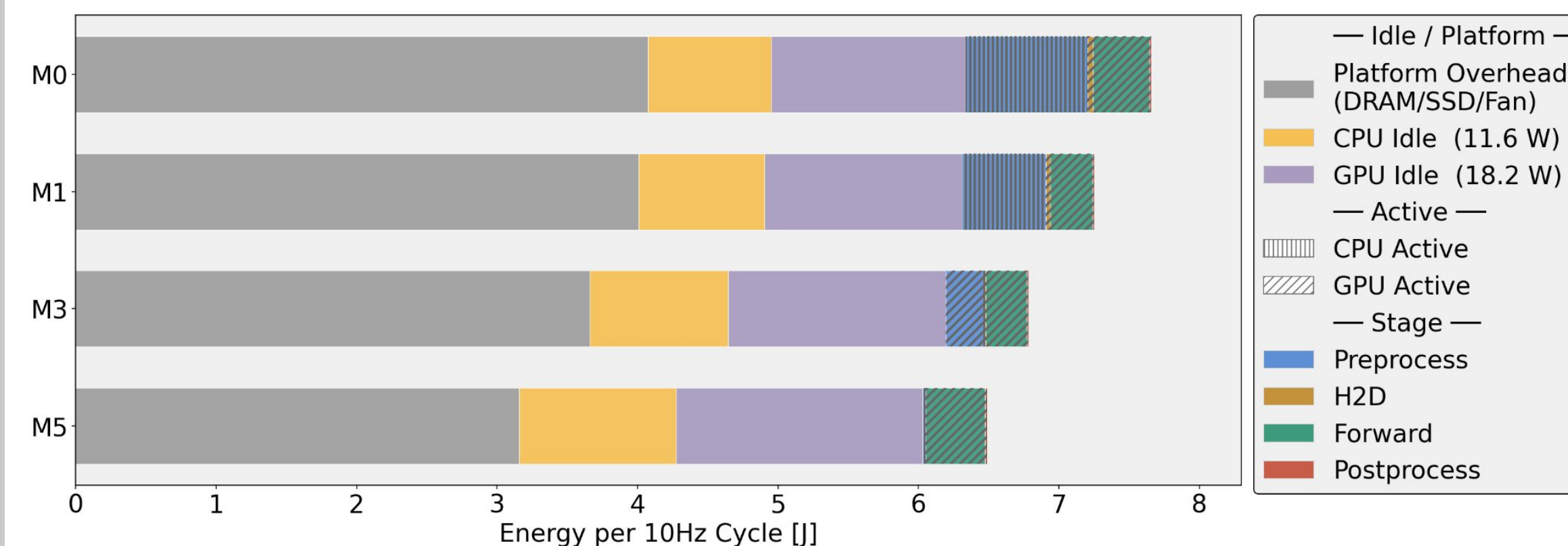
Energy (NVIDIA A10, GPU-Only)



- Measured with NVIDIA Management Library (NVML), calculated/estimated preprocess
- M1 (Compiled) and AMP save energy
- Measurement Bias:** M3/M4 consumes more energy per sample in GPU-only metrics (CPU energy not captured)
- Need holistic, full-system energy analysis

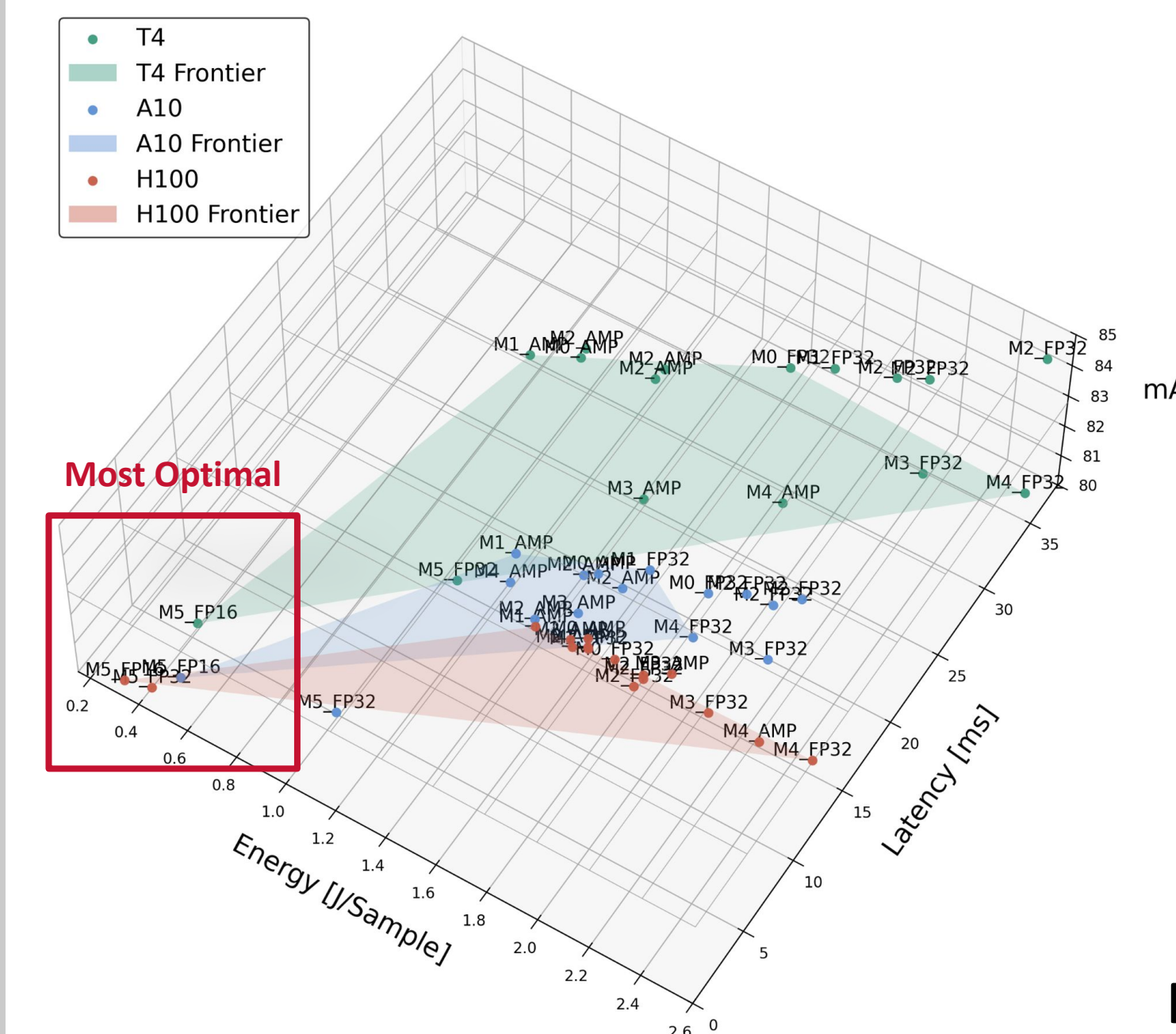
Evaluation

Full-System Energy Breakdown (RTX 3080 Ti, Intel RAPL, 10Hz Cycle)



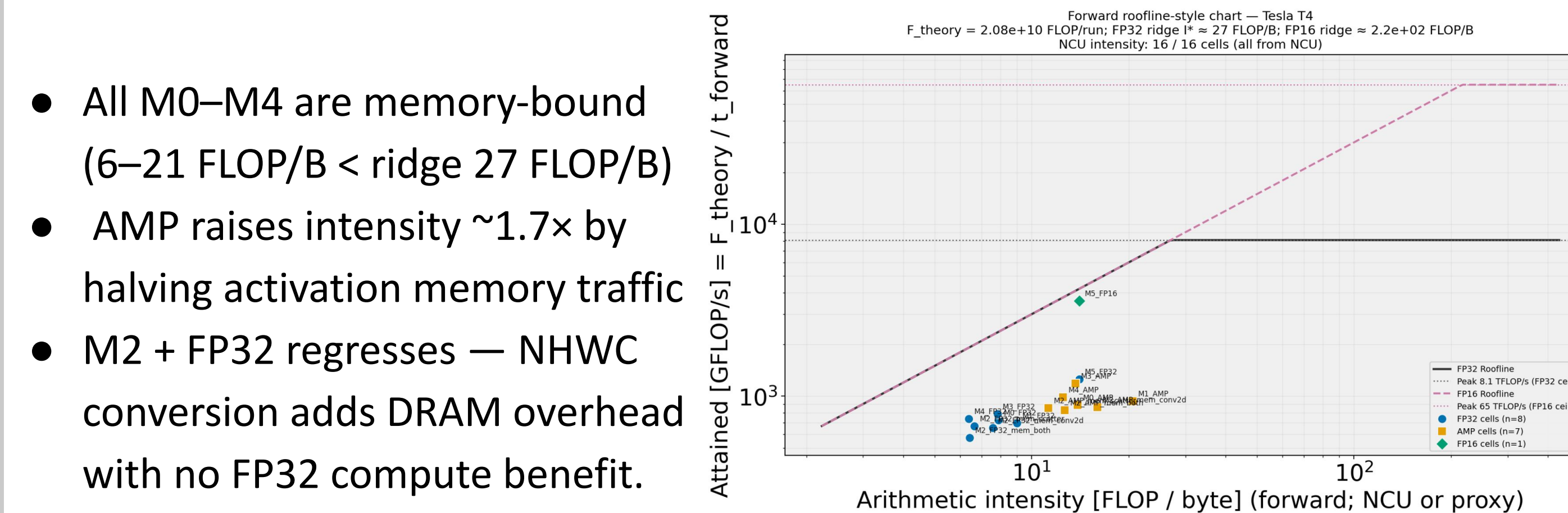
- Reduced inference time -> reduced active energy cost (CPU: 78W, GPU: 19.4W) with increased idle energy (CPU: 11.6W, GPU: 18.2W)
- M3 (Voxelization offloaded) replaces CPU compute (~78 W) with GPU execution (~19.4 W) -> net system energy reduction

Hardware Scaling (NVIDIA T4, A10, H100)



- Accuracy-latency-energy pareto frontier across GPUs
- Largest improvement on T4 (-75% J/sample, -83% latency) with M5
- Optimizations narrow performance disparity across hardware

Roofline Analysis (NVIDIA T4)



- All M0–M4 are memory-bound (6–21 FLOP/B < ridge 27 FLOP/B)
- AMP raises intensity ~1.7× by halving activation memory traffic
- M2 + FP32 regresses — NHWC conversion adds DRAM overhead with no FP32 compute benefit.

Conclusion

- Our optimized 3D perception pipeline cuts latency by **79%** and energy cost by **73%** without significant accuracy degradation on A10 GPU.
- Hardware scaling analysis shows **largest improvement on resource-constrained GPUs** (critical for robotics).
- Optimizations are **deeply coupled** (e.g. NHWC & AMP).
- Real deployment needs **hardware-aware automatic methods** to lower engineering cost.